

1. Project overview

This project implements a decentralized blood marketplace on Ethereum where:

- **Donors** register blood donations, which are tokenized as ERC-1155 tokens (one token ID per blood unit or homogeneous batch).[openzeppelin+1](#)
- **Buyers** (any Ethereum address: individual, hospital, etc.) purchase those units using an ERC-20 token named **YODA**.
- The system's goal is to demonstrate that blood units can be represented and transacted as on-chain digital assets with attached metadata, not to model real-world medical or regulatory constraints.

Key constraints and simplifications:

- No logistics, clinical verification, or regulator logic on-chain.
- No special institutional roles (no collection centers, hospitals, regulators).
- You trust whatever metadata the donor provides; there is no on-chain validation of blood type, screening, or storage.
- There is an **admin** (contract owner) who can configure parameters but does not intervene in normal flows.

Core components:

1. **YodaToken** – ERC-20 fungible token used for payments.
 2. **BloodRegistry** – ERC-1155 token contract that represents blood units and stores their metadata.
 3. **BloodMarket** – marketplace contract that lets donors list blood tokens and buyers purchase them with YODA.
-

2. High-level architecture

2.1 On-chain components

- **YodaToken (ERC-20)**
 - Standard fungible token used as the settlement currency for trades.[soliditylang+1](#)
- **BloodRegistry (ERC-1155)**
 - Mints new blood tokens when donors register units.
 - Stores per-token metadata (blood type, volume, timestamps, etc.).
 - Tracks high-level status (Available, Sold, etc.).[openzeppelin+1](#)
- **BloodMarket**

- Maintains listings: offers from donors to sell a given token ID and amount at a given price in YODA.
- Executes trades: transfers YODA from buyer to seller and transfers ERC-1155 blood tokens from seller to buyer.dev+1

2.2 Off-chain components (out of scope to implement, but relevant)

- dApp/UI for donors and buyers (e.g., React + ethers.js).
 - Optionally an indexer (e.g., The Graph) to provide fast search/filtering over listings and blood metadata.zypsy+1
-

3. Roles and trust model

Only one privileged role is needed: **admin**.

- **Admin (contract owner):**
 - Deploys contracts.
 - Configures basic parameters (market address, fee configuration, URI template).
 - Can pause/unpause contracts in emergencies (optional).

All other users are unprivileged EOAs:

- **Donor:** any address that calls `registerUnit` on `BloodRegistry`.
- **Buyer:** any address that calls `buy` on `BloodMarket` and pays YODA.

There is no on-chain enforcement that the buyer is a hospital or that the donor is medically eligible.

4. Token standards and libraries

- **ERC-20** for YODA. Use OpenZeppelin's implementation to reduce risk and boilerplate.soliditylang+1
- **ERC-1155** for multi-token representation of blood units. This allows efficient management of many token IDs and batch transfers within a single contract.openzeppelin+1
- **Ownable** (or `AccessControl` if you prefer) for admin privileges.
- **ReentrancyGuard** and `SafeERC20` for secure payment flows.jamesbachini+1

5. Data model

5.1 Blood unit metadata (**BloodRegistry**)

Each ERC-1155 `tokenId` corresponds to one blood unit or a homogeneous batch. Metadata:

```
enum Status {
    Available,
    Reserved,    // optional, you can skip if not used
    Sold,
    Transfused,
    Expired,
    Discarded
}

struct BloodInfo {
    // Medical-ish attributes (trusted input from donor)
    uint8  bloodGroup;    // 0:A,1:B,2:AB,3:O
    bool   rhPositive;    // true = Rh+, false = Rh-
    uint8  component;     // 0:Whole,1:PRBC,2:Platelets,3:Plasma,...
    uint16 volumeMl;      // approximate volume in ml

    // Time-related
    uint64 collectionTime; // block timestamp when registered
    uint64 expiryTime;     // donor-specified expiry

    // Additional context
    string storageDetails; // free-form string (e.g., "Home freezer"
/ "N/A")
    uint8  priority;      // arbitrary 0-255 value indicating
urgency

    // Lifecycle / ownership
    Status status;        // starts as Available
    address donor;        // original owner at registration
}
```

Storage:

```
uint256 public nextTokenId;
mapping(uint256 => BloodInfo) public bloodInfo;
address public market; // optional: to allow market to change status
```

5.2 Listings (BloodMarket)

```
struct Listing {
    uint256 id;
    address seller;           // current owner (typically donor)
    uint256 tokenId;         // blood token ID in BloodRegistry
    uint256 amount;          // number of units up for sale
    uint256 pricePerUnit;    // YODA (wei) per unit
    uint64  createdAt;
    bool    active;
}
```

Storage:

```
IERC20 public immutable yoda;
BloodRegistry public immutable registry;

uint256 public nextListingId;
mapping(uint256 => Listing) public listings;

// Optional protocol fee
uint16 public protocolFeeBps; // basis points, e.g., 100 = 1%
address public feeRecipient;
```

6. Contract specifications

6.1 YodaToken (ERC-20)

Goal: fungible token for payments and potential incentives.

Inheritance:

- `ERC20`
- `Ownable` (for minting control).

State:

- No extra state beyond ERC-20; optionally track `MINTER_ROLE` if you use `AccessControl`.

Key functions:

1. `constructor(string memory name_, string memory symbol_)`
 - Sets token name and symbol.
 - Mints initial supply to admin or leaves supply at 0 and lets admin mint later.
 2. `function mint(address to, uint256 amount) external onlyOwner`
 - Used to distribute YODA for testing or bootstrap.
 3. Standard ERC-20 functions (`transfer`, `approve`, `transferFrom`).
-

6.2 BloodRegistry (ERC-1155)

Goal: represent blood donations as ERC-1155 tokens, track their metadata and high-level status.

Inheritance:

- `ERC1155`
- `Ownable`
- Optional: `Pausable`, `ERC1155Burnable`.

State:

- `uint256 public nextTokenId;`
- `mapping(uint256 => BloodInfo) public bloodInfo;`
- `address public market;` – set once `BloodMarket` is deployed.

Events:

- `event UnitRegistered(uint256 indexed tokenId, address indexed donor, BloodInfo info);`
- `event StatusUpdated(uint256 indexed tokenId, Status previous, Status current);`

Functions:

1. `constructor(string memory uri_) ERC1155(uri_)`
 - `uri_` may be a generic template; you can return metadata via off-chain API or leave it as is.
2. `function setMarket(address market_) external onlyOwner`
 - Sets the marketplace contract address; optional.
3. `function registerUnit(uint8 bloodGroup, bool rhPositive, uint8 component, uint16 volumeMl, uint64 expiryTime, string calldata storageDetails, uint8 priority, uint256 amount) external returns (uint256 tokenId)`
 - Caller is treated as the donor: `address donor = msg.sender;`
 - Requirements:
 - `amount > 0.`
 - `volumeMl > 0.`
 - `expiryTime > block.timestamp.`
 - Behavior:
 - `tokenId = nextTokenId++.`
 - Populate `BloodInfo` with:
 - `collectionTime = uint64(block.timestamp).`
 - `status = Status.Available.`
 - `donor = donor.`
 - Call `_mint(donor, tokenId, amount, "")`.
 - Emit `UnitRegistered(tokenId, donor, info)`.
4. `function updateStatus(uint256 tokenId, Status newStatus) external`
 - For this simplified project:
 - Either restrict to `msg.sender == market` **or** allow the current holder of the token to change status.
 - You can start with no enforced transition rules (just log status changes) to keep it simple.
5. `function isExpired(uint256 tokenId) external view returns (bool)`
 - `return block.timestamp > bloodInfo[tokenId].expiryTime;`
6. Standard ERC-1155 functions (already implemented in OpenZeppelin):
 - `balanceOf`, `balanceOfBatch`, `safeTransferFrom`, `setApprovalForAll`, etc.`openzeppelin+1`

6.3 BloodMarket (marketplace)

Goal: allow any holder of a blood token to create a listing, and any buyer to purchase a portion or all of that listing using YODA.

Inheritance:

- Ownable
- ReentrancyGuard

State:

- IERC20 public immutable yoda;
- BloodRegistry public immutable registry;
- uint256 public nextListingId;
- mapping(uint256 => Listing) public listings;
- Optional:
 - uint16 public protocolFeeBps;
 - address public feeRecipient;

Events:

- event ListingCreated(uint256 indexed id, address indexed seller, uint256 indexed tokenId, uint256 amount, uint256 pricePerUnit);
- event ListingUpdated(uint256 indexed id, uint256 newAmount);
- event ListingCancelled(uint256 indexed id);
- event ListingFilled(uint256 indexed id, address indexed buyer, uint256 amount, uint256 totalPaid);

Functions:

1. constructor(address yoda_, address registry_)
 - Store yoda = IERC20(yoda_); and registry = BloodRegistry(registry_);.
 - Set owner to deployer.
2. function setFees(uint16 protocolFeeBps_, address feeRecipient_) external onlyOwner
 - Optional fees; ensure protocolFeeBps_ <= 10_000.
3. `function createListing(uint256 tokenId,

```
uint256 amount,  
uint256 pricePerUnit  
) external nonReentrant returns (uint256 listingId)`
```

- Requirements:
 - `amount > 0.`
 - `pricePerUnit > 0.`
 - `registry.balanceOf(msg.sender, tokenId) >= amount.`
 - Behavior:
 - `listingId = nextListingId++.`
 - Create `Listing` with:
 - `seller = msg.sender.`
 - `createdAt = uint64(block.timestamp).`
 - `active = true.`
 - Emit `ListingCreated.`
 - Note: This does not transfer tokens yet; relies on `setApprovalForAll` for ERC-1155 transfers later, which is standard marketplace design.[openzeppelin+1](#)
4. `function cancelListing(uint256 listingId) external nonReentrant`
- Requirements:
 - `msg.sender == listings[listingId].seller.`
 - `listings[listingId].active == true.`
 - Behavior:
 - Set `active = false.`
 - Emit `ListingCancelled.`
5. `function buy(uint256 listingId, uint256 amount) external nonReentrant`
- Requirements:
 - `amount > 0.`
 - `listing.active == true.`
 - `amount <= listing.amount.`
 - Optionally: `!registry.isExpired(listing.tokenId).`
 - Calculations:
 - `uint256 total = amount * listing.pricePerUnit;`
 - `uint256 fee = total * protocolFeeBps / 10_000;`
 - `uint256 sellerAmount = total - fee;`
 - Transfers:
 - `yoda.transferFrom(msg.sender, listings[listingId].seller, sellerAmount);`
 - If `fee > 0`, `yoda.transferFrom(msg.sender, feeRecipient, fee);`

- `registry.safeTransferFrom(listings[listingId].seller, msg.sender, listing.tokenId, amount, "");`
 - Updates:
 - `listings[listingId].amount -= amount;`
 - If `listings[listingId].amount == 0`, set `active = false`.
 - Optional:
 - Call `registry.updateStatus(tokenId, Status.Sold)` for logging.
 - Emit `ListingFilled(listingId, msg.sender, amount, total);`
-

7. End-to-end workflows

7.1 Donor registers a donation

1. Donor connects wallet to dApp.
2. Fills form fields:
 - `bloodGroup, rhPositive, component, volumeMl, expiryTime, storageDetails, priority, amount`.
3. dApp calls `BloodRegistry.registerUnit(...)`.
4. `BloodRegistry` mints `amount` tokens of new `tokenId` to donor.
5. Frontend shows donor their new unit(s) with metadata.

7.2 Donor lists a token for sale

1. Donor calls `BloodRegistry.setApprovalForAll(bloodMarketAddress, true)`.
2. Donor calls `BloodMarket.createListing(tokenId, amount, pricePerUnit)`.
3. Listing appears in UI as active.

7.3 Buyer purchases a listing

1. Buyer obtains YODA (via faucet or admin mint in dev).
2. Buyer calls `YodaToken.approve(BloodMarket, totalYodaNeeded)`.
3. Buyer calls `BloodMarket.buy(listingId, amount)`.
4. `BloodMarket`:
 - Moves YODA from buyer to seller (and fee recipient if configured).
 - Transfers ERC-1155 blood tokens to buyer.

5. Listing remaining amount decreases; if 0, listing becomes inactive.
 6. Buyer now owns those blood tokens (proof of having acquired the units).
-

8. Non-functional requirements and coding guidelines

- Use **OpenZeppelin** implementations for ERC-20, ERC-1155, Ownable, and ReentrancyGuard to avoid common bugs.openzeppelin+1
 - Follow Solidity style guides (naming, error messages, checks-effects-interactions pattern).rareskills+1
 - Use **nonReentrant** on marketplace functions that move funds and tokens.dev+1
 - Use **require** with clear revert messages for all preconditions.
 - Add NatSpec comments (**/// @notice**, **/// @dev**, **@param**, **@return**) on all public/external functions to make the code self-documenting.
-

9. Testing checklist

For the engineer, at minimum:

1. **YodaToken**
 - Minting by owner, transfers, approvals.
2. **BloodRegistry**
 - **registerUnit** creates a new tokenId and mints to donor.
 - Metadata is stored correctly.
 - **isExpired** returns correct values before/after expiry.
 - Transfers of ERC-1155 tokens between accounts.
3. **BloodMarket**
 - **createListing** fails if not enough balance or incorrect params.
 - **cancelListing** fails if not seller; succeeds otherwise.
 - **buy**:
 - Reverts if listing inactive, **amount** too large, or insufficient YODA allowance.
 - Correctly transfers YODA and ERC-1155 tokens.
 - Updates listing amount and active flag.
 - Fee calculation correctness if protocol fees are enabled.